PROJECT REPORT ON

Freelancer Project Management and Payment Tracking System

Submitted by

SUMRETTHA G

25BIT157

Under the Guidance of

_____

_____

Technical Department, KG College of Arts and Science

Academic Year: 2025 – 2026

APRIL 2026

Semester: I

DECLARATION

I hereby declare that the project entitled “Freelancer Project Management and Payment Tracking System” submitted in partial fulfillment of the requirements for the course _____ is a record of my original work carried out under the guidance of _____, Technical Trainer, Technical Department, KG College of Arts and Science.

I further declare that this project has not been submitted previously, either in part or in full, for the award of any degree, diploma, or other similar title, at this or any other institution.

I have acknowledged all sources of information used in this project wherever applicable.

April 2026

Coimbatore

_____

(Your Name)

CERTIFICATE

This is to certify that the project entitled “Freelancer Project Management and Payment Tracking System” is a bona fide work carried out by SUMRETTHA G _____, _____, KG College of Arts & Science, in partial fulfillment of the requirements for the course _____

This project has been completed under my supervision and guidance during the academic period April 2026.

Place: _____

Date: _____

Submitted for the Viva-Voce Examination held on _____

Guide: _____ Head of the Department: _____

(Technical Trainer, KGCAS) (Technical Head, KGCAS)

TABLE OF CONTENTS

S. No.	Topics	Page. No.
1	Introduction	
1.1	Objective	
1.2	Overview	
1.3	Features	
2	Program Analysis	
2.1	Problem Definition	
2.2	Project Overview	
2.3	Scopes and Limitations	
3	System Design	
3.1	Modules	
4	Methodology	
4.1	Problem Analysis	
4.2	System Design	
4.3	Program Development	
4.4	Testing and Validation	
4.5	Result and Documentation	
4.6	Algorithm	
4.7	Flowchart	
5	Source Code	
6	Input and Output Screens	
7	Discussion and Conclusion	
7.1	Discussion	
7.2	Conclusion	
8	References	

ABSTRACT

The Freelancer Project Management and Payment Tracking System is a software application designed to streamline the management of freelance projects and automate payment tracking between freelancers and clients. In many freelance engagements, project coordination and payment handling are performed manually through emails, spreadsheets, or informal communication, which often leads to missed deadlines, payment disputes, and inefficient record keeping. This project aims to provide a centralized platform that simplifies project management while ensuring transparent and accurate payment tracking.

The system allows freelancers to create and manage projects, assign tasks, monitor progress, and record deliverables. Clients can review project milestones, approve completed work, and track payments against invoices. The application automatically calculates payment amounts based on agreed rates, milestones, or hourly work, and generates structured invoices including applicable service charges or taxes.

Developed using programming concepts such as object-oriented design and modular architecture, the system can be implemented as a web-based or desktop application. Core functionalities include project creation, task tracking, invoice generation, payment ledger management, and reporting.

The main objective of the system is to improve efficiency in freelance project management, reduce conflicts over payments, and provide a transparent workflow for both freelancers and clients. By automating repetitive tasks and ensuring accurate financial records, the system enhances productivity and builds trust between stakeholders.

This project demonstrates the practical application of software engineering principles in solving real-world challenges faced by freelancers and lays the foundation for more advanced freelance marketplace and management systems in the future.

1. INTRODUCTION

Freelancing has become a vital part of the global workforce, offering flexibility and opportunities for both professionals and clients. However, managing freelance projects and tracking payments often remains a challenge due to reliance on informal tools such as emails, spreadsheets, or chat applications. These methods can lead to inefficiencies, missed deadlines, and disputes over payments.

The Freelancer Project Management and Payment Tracking System is designed to address these issues by providing a centralized platform that integrates project management with financial tracking. The system ensures transparency, accuracy, and efficiency in handling freelance engagements, benefiting both freelancers and clients.

1.1 Objective

• To automate project assignment, task monitoring, and milestone tracking.

• To provide a secure and reliable method for recording payments and generating invoices.

• To reduce conflicts between freelancers and clients by maintaining clear records of deliverables and payments.

• To enhance productivity by minimizing manual effort in project coordination and financial management.

• To build trust through transparency and accountability in freelance work.

1.2 Overview

The system offers a structured approach to freelance project management by combining task tracking with payment monitoring. Freelancers can create projects, assign tasks, and monitor progress, while clients can review deliverables, approve milestones, and track payments.

• Project Management Module: For creating projects, assigning tasks, and monitoring deadlines.

• Payment Tracking Module: For recording transactions, calculating amounts due, and generating invoices.

• Reporting Tools: For analyzing project progress and financial records.

• User-Friendly Interface: Designed to be intuitive for both freelancers and clients.

1.3 Features

The Freelancer Project Management and Payment Tracking System is designed with a set of core features that ensure efficiency, transparency, and ease of use for both freelancers and clients.

• Project Creation and Management – Freelancers can create new projects, define objectives, and break them down into tasks or milestones. Clients can view project details, monitor progress, and approve deliverables.

• Task Tracking and Deadlines – Each project can be divided into tasks with assigned deadlines. Progress tracking ensures timely completion and accountability.

• Payment Tracking and Invoice Generation – The system records payments against milestones or hourly work. Automated invoice generation includes itemized tasks, rates, taxes, and final payable amounts.

• Secure Ledger System – Maintains a transparent record of all transactions between freelancers and clients and reduces disputes by providing clear financial documentation.

• Reporting and Analytics – Generates reports on project progress, completed tasks, and payment history.

• User-Friendly Interface – Simple navigation for freelancers to manage projects and for clients to track payments.

• Scalability and Flexibility – Supports multiple projects and clients simultaneously and can be extended to integrate with online payment gateways or freelance marketplaces.

2. PROGRAM ANALYSIS

Freelancers and clients often face challenges in managing projects and tracking payments due to the lack of a centralized system. Manual methods such as emails, spreadsheets, or chat applications are prone to errors, inefficiencies, and disputes. This section analyzes the core problems, provides an overview of the project, and examines existing systems.

2.1 Problem Definition

• Lack of Transparency: Freelancers and clients often struggle to maintain clear records of deliverables and payments.

• Manual Tracking: Using spreadsheets or emails leads to errors, missed deadlines, and payment delays.

• Disputes Over Payments: Without structured invoices or records, disagreements arise regarding the amount due or work completed.

• Inefficient Communication: Project updates and approvals are often scattered across multiple platforms, reducing efficiency.

• Limited Reporting: Freelancers lack tools to analyze project progress and financial performance.

2.2 Project Overview

The Freelancer Project Management and Payment Tracking System is designed to solve these issues by providing a centralized platform that integrates project management with financial tracking.

• Freelancers to create projects, assign tasks, and monitor progress.

• Clients to review deliverables, approve milestones, and track payments.

• Automated invoice generation with itemized tasks, rates, taxes, and final payable amounts.

• Secure storage of project and payment records to reduce disputes.

• Reporting tools to analyze efficiency and financial performance.

2.3 Scopes and Limitations

• Scope: Managing freelance projects and tasks.

• Scope: Tracking deadlines and milestones.

• Scope: Recording payments and generating invoices.

• Scope: Providing reports on project progress and financial records.

• Scope: Practical use of software engineering and programming concepts.

• The system can be used in freelance work management, small businesses hiring freelancers, agencies coordinating multiple freelancers, individual freelancers handling multiple clients, and startups and project-based teams.

• Limitations: No direct online payment gateway in the basic version.

• Limitations: Limited customization for invoices and reports.

• Limitations: Accuracy depends on correct data entry.

• Limitations: Basic security features only.

• Limitations: Requires internet if web-based.

• Limitations: Best suited for small to medium projects.

3. SYSTEM DESIGN

System design is the stage where the structure and components of the system are planned before implementation. In this phase, the system is divided into different modules, and the workflow of the application is defined. Proper system design helps in organizing the program efficiently and makes development easier.

The Freelancer Project Management and Payment Tracking System is designed as a task-driven and finance-linked application where freelancers can manage projects, track progress, and automatically generate invoices linked to milestones.

3.1 Modules

• 1. Project Creation & Assignment Module – Allows freelancers to create new projects, define scope, deadlines, and milestones, and assign tasks to self or team members.

• 2. Task Management Module – Displays all tasks with status (Pending, In Progress, Completed), supports priority levels and deadlines, and provides reminders and progress tracking.

• 3. Client Communication Module – Enables secure messaging between freelancer and client, supports file sharing (contracts, deliverables), and maintains a record of discussions for transparency.

• 4. Payment Tracking Module – Records invoices linked to project milestones, tracks payment status (Pending, Paid, Overdue), and supports multiple payment methods (Bank, PayPal, UPI).

• 5. Invoice Generation Module – Automatically generates invoices with client details, project name, milestone, and amount; calculates applicable taxes (e.g., GST); and provides downloadable PDF invoices.

• 6. Analytics & Reporting Module – Generates reports on earnings per client/project, tracks overdue payments and upcoming deadlines, and provides financial summaries for tax filing.

• 7. Exit Module – Allows the user to safely log out or terminate the application after completing tasks and billing.

4. METHODOLOGY

Methodology describes the process followed to develop the project. It explains the steps taken from understanding the problem to designing, implementing, testing, and documenting.

The development process includes Problem Analysis, System Design, Program Development, Testing and Validation, and Result and Documentation.

4.1 Problem Analysis

• Freelancers often face challenges in managing multiple projects and tracking payments.

• Issues include missed deadlines, delayed payments, and lack of financial transparency.

• The system aims to solve these problems by integrating project management with payment tracking in one platform.

• Based on these problems, the system requirements were defined to develop an automated billing application that simplifies project management and payment tracking.

4.2 System Design

• The system is modular, combining project management and payment tracking.

• Each module (Project Creation, Task Management, Communication, Payment Tracking, Invoice Generation, Analytics, Exit) is designed to handle a specific function.

• Workflow ensures that project milestones trigger invoice generation and payment tracking automatically.

4.3 Program Development

• The supplied source code is implemented in C++ using object-oriented programming concepts.

• The program uses a Project class, an array of Project objects, methods, loops, conditional statements, switch-case selection, and basic input validation.

• The program stores project details and payment information in memory and provides menu-driven operations.

4.4 Testing and Validation

• Unit Testing: Each module tested individually.

• Integration Testing: Ensures modules work together (project creation → payment update → summary).

• Validation: Cross-checks budget, received payment, and pending payment values against expected results.

• User Testing: Sample project details and payment values can be entered to confirm usability.

4.5 Result and Documentation

• The system successfully manages projects and links them to payments.

• Users can view projects and calculate pending payments from stored project data.

• Documentation includes the source code, test cases, and report sections.

4.6 Algorithm

• 1. Start the program.

• 2. Display the Freelancer Project Management menu.

• 3. Add a project by entering project name, client name, budget, payment, and status.

• 4. Validate budget and payment values.

• 5. View all stored projects and display pending payment.

• 6. Update payment for a selected project after validating the new amount.

• 7. Calculate total budget, total received, and total pending payment.

• 8. Repeat until the user selects Exit.

• 9. Stop.

4.7 Flowchart

Start → Display Menu → Add / View / Update Payment / Payment Summary → Perform Selected Operation → Return to Menu → Exit.

5. SOURCE CODE

The following C++ source code is inserted as the updated implementation for the Freelancer Project Management and Payment Tracking System. It provides project creation, project viewing, payment updating, and payment summary functions.

```
#include <iostream>
#include <string>
#include <iomanip>
using namespace std;

class Project {
public:
    int id;
    string projectName, clientName, status;
    double budget, paid;

    void addProject(int projectId) {
        id = projectId;

        cin.ignore();

        cout << "Enter Project Name: ";
        getline(cin, projectName);

        cout << "Enter Client Name: ";
        getline(cin, clientName);

        cout << "Enter Project Budget: ";
        cin >> budget;

        cout << "Enter Payment Received: ";
        cin >> paid;

        if (budget <= 0 || paid < 0 || paid > budget) {
            cout << "Invalid budget or payment!\n";
            budget = 0;
            paid = 0;
            return;
        }

        int choice;
        cout << "\n1. Pending";
        cout << "\n2. In Progress";
        cout << "\n3. Completed";
        cout << "\nEnter Project Status: ";
        cin >> choice;

        switch (choice) {
            case 1: status = "Pending"; break;
            case 2: status = "In Progress"; break;
            case 3: status = "Completed"; break;
            default: status = "Pending";
        }

        cout << "\nProject Added Successfully!\n";
    }

    void display() {
        cout << "\nProject ID: " << id;
        cout << "\nProject Name: " << projectName;
        cout << "\nClient Name: " << clientName;
        cout << "\nProject Budget: Rs. " << budget;
        cout << "\nPayment Received: Rs. " << paid;
        cout << "\nPayment Pending: Rs. " << budget - paid;
        cout << "\nProject Status: " << status;
        cout << "\n----------------------------\n";
    }
};

int main() {
```

```
Project p[100];
int count = 0;
int choice;

do {
    cout << "\n===================================";
    cout << "\n FREELANCER PROJECT MANAGEMENT";
    cout << "\n===================================";
    cout << "\n1. Add Project";
    cout << "\n2. View All Projects";
    cout << "\n3. Update Payment";
    cout << "\n4. Payment Summary";
    cout << "\n5. Exit";
    cout << "\nEnter Your Choice: ";
    cin >> choice;

    switch (choice) {
        case 1:
            if (count < 100) {
                p[count].addProject(count + 1);
                if (p[count].budget > 0)
                    count++;
            } else {
                cout << "Project storage is full!\n";
            }
            break;

        case 2:
            if (count == 0) {
                cout << "No Projects Available!\n";
            }
            for (int i = 0; i < count; i++) {
                p[i].display();
            }
            break;

        case 3: {
            int id;
            double payment;
            bool found = false;

            cout << "Enter Project ID: ";
            cin >> id;

            for (int i = 0; i < count; i++) {
                if (p[i].id == id) {
                    cout << "Enter Total Payment Received: ";
                    cin >> payment;

                    if (payment >= p[i].paid &&
                        payment <= p[i].budget) {
                        p[i].paid = payment;
                        cout << "Payment Updated Successfully!\n";
                    } else {
                        cout << "Invalid Payment Amount!\n";
                    }

                    found = true;
                    break;
                }
            }

            if (!found)
                cout << "Project Not Found!\n";
```

```
                break;
            }

            case 4: {
                double totalBudget = 0, totalPaid = 0;

                for (int i = 0; i < count; i++) {
                    totalBudget += p[i].budget;
                    totalPaid += p[i].paid;
                }

                cout << fixed << setprecision(2);
                cout << "\nTotal Projects: " << count;
                cout << "\nTotal Budget: Rs. " << totalBudget;
                cout << "\nTotal Received: Rs. " << totalPaid;
                cout << "\nTotal Pending: Rs. "
                     << totalBudget - totalPaid << "\n";
                break;
            }

            case 5:
                cout << "Thank You!\n";
                break;

            default:
                cout << "Invalid Choice! Try Again.\n";
        }

    } while (choice != 5);

    return 0;
}
```

6. INPUT AND OUTPUT SCREENS

The program is menu-driven and accepts project details through the console. Typical inputs include project name, client name, project budget, payment received, project status, project ID, and updated payment amount.

The corresponding output displays project information, payment pending for each project, success or validation messages, and an overall payment summary.

7. DISCUSSION & CONCLUSION

7.1 Discussion

The Freelancer Project Management and Payment Tracking System was developed to help freelancers manage projects and track payments in one place. The updated implementation uses a Project class and menu-driven operations for project creation, project display, payment update, and payment summary.

During development, C++ programming concepts were used to code the main logic, including classes, objects, methods, arrays, loops, conditional statements, switch-case selection, and basic input validation.

The system can be tested with different project names, client names, budgets, payment values, and statuses to ensure that it performs the required functions effectively. The program reduces manual effort and helps freelancers stay organized while tracking payment information.

7.2 Conclusion

The Freelancer Project Management and Payment Tracking System demonstrates the practical application of programming concepts to a real-world project and payment tracking problem. The system supports project creation, viewing, payment updating, and calculation of pending payments.

The project helps in understanding object-oriented programming and menu-driven application development. The system can be further enhanced with a graphical user interface, database storage, multiple currency support, cloud-based project and invoice storage, and integration with online payment gateways.

8. REFERENCES

Books

• Herbert Schildt – Java: The Complete Reference

Websites

• Oracle Java documentation

• GeeksforGeeks

• W3Schools

• JavaTpoint